# ADVANCED SQL QUERIES USING JOINS AND SUBQUERIES

ASSIGNMENT 9

DATABASE MANAGEMENT SYSTEM

ALLIANCE UNIVERSITY

Department of Computer Applications

SUMMITED BY: BHUVANA S

REGISTER NO: 2511021240036

COURSE: BCA

YEAR: 2ND YEAR

# Introduction

Advanced SQL queries help retrieve meaningful information from multiple related tables in a database. Joins combine records from different tables using related columns, while subqueries use the result of one query within another query. In this assignment, joins and subqueries are implemented in the Food Delivery Management System using MySQL to generate reports about customer orders, menu items, restaurants, and order amounts.

## *1. Report on Customer Orders Using INNER JOIN*

The INNER JOIN combines records from two or more tables when their related column values match.

```
SELECT
    O.Order_ID,
    C.Customer_Name,
    R.Restaurant_Name,
    O.Order_Date,
    O.Total_Amount,
    O.Order_Status
FROM ORDERS O
INNER JOIN CUSTOMER C
    ON O.Customer_ID = C.Customer_ID
INNER JOIN RESTAURANT R
    ON O.Restaurant_ID = R.Restaurant_ID;
```

| Order_ID | Customer_Name | Restaurant_Name | Order_Date | Total_Amount | Order_Status |
|---|---|---|---|---|---|
| 401 | Arun Kumar | Spice Garden | 2026-10-15 | 260.00 | Delivered |
| 402 | Priya Sharma | Green Bowl | 2026-10-16 | 150.00 | Delivered |
| 403 | Rahul Verma | Food Junction | 2026-10-17 | 160.00 | Delivered |
| 404 | Sneha Reddy | Urban Tandoor | 2026-10-18 | 280.00 | Preparing |
| 405 | Karthik Raj | South Kitchen | 2026-10-19 | 120.00 | Confirmed |

This query joins the ORDERS, CUSTOMER, and RESTAURANT tables to display each order with the customer's name and the restaurant's name. It helps track orders and identify the customers and restaurants associated with them.

## *2. Report on Ordered Food Items Using Multiple Joins*

This report uses joins to display the food items included in each order, along with their quantities and subtotals.

```
SELECT
    O.Order_ID,
    C.Customer_Name,
    M.Item_Name,
    OI.Quantity,
    OI.Subtotal
FROM ORDERS O
INNER JOIN CUSTOMER C
    ON O.Customer_ID = C.Customer_ID
INNER JOIN ORDER_ITEM OI
    ON O.Order_ID = OI.Order_ID
INNER JOIN MENU_ITEM M
    ON OI.Item_ID = M.Item_ID
ORDER BY O.Order_ID, OI.Order_Item_ID;
```

| Order_ID | Customer_Name | Item_Name | Quantity | Subtotal |
|---|---|---|---|---|
| 401 | Arun Kumar | Paneer Biryani | 1 | 220.00 |
| 401 | Arun Kumar | Butter Naan | 1 | 40.00 |
| 402 | Priya Sharma | Vegetable Salad | 1 | 150.00 |
| 403 | Rahul Verma | Masala Dosa | 2 | 160.00 |
| 404 | Sneha Reddy | Chicken Tandoori | 1 | 280.00 |
| 405 | Karthik Raj | Idli Sambar | 2 | 120.00 |

This query joins four tables to show which food items each customer ordered. It also displays the quantity and subtotal of every item. This report is useful for checking order contents and preparing order details.

### *3. Report on Restaurants and Their Menu Item Counts Using LEFT JOIN*

The LEFT JOIN returns all records from the left table, including those without a matching record in the right table.

```
SELECT
   R.Restaurant_ID,
   R.Restaurant_Name,
   COUNT(M.Item_ID) AS Total_Menu_Items
FROM RESTAURANT R
LEFT JOIN MENU_ITEM M
   ON R.Restaurant_ID = M.Restaurant_ID
GROUP BY R.Restaurant_ID, R.Restaurant_Name
ORDER BY R.Restaurant_ID;
```

| Restaurant_ID | Restaurant_Name | Total_Menu_Items |
|---|---|---|
| 101 | Spice Garden | 2 |
| 102 | Green Bowl | 2 |
| 103 | Food Junction | 1 |
| 104 | Urban Tandoor | 2 |
| 105 | South Kitchen | 1 |

This query displays every restaurant and counts the menu items associated with it. LEFT JOIN ensures that a restaurant would still appear even if it had no menu items. GROUP BY calculates the count separately for each restaurant.

### *4. Report on Menu Items Priced Above the Average Using a Subquery*

A subquery is a query placed inside another SQL query. The inner query calculates the average menu item price, and the outer query selects items priced above that average.

```
SELECT Item_ID, Item_Name, Price
FROM MENU_ITEM
WHERE Price > (
```

```
    SELECT AVG(Price)
    FROM MENU_ITEM
)
ORDER BY Price DESC;
```

| Item_ID | Item_Name | Price |
|---|---|---|
| 305 | Chicken Tandoori | 280.00 |
| 301 | Paneer Biryani | 220.00 |
| 308 | Veg Fried Rice | 170.00 |
| 303 | Vegetable Salad | 150.00 |
| NULL | NULL | NULL |

The subquery calculates the average price of all eight menu items, which is ₹140. The outer query displays only the items priced above ₹140, sorted from highest to lowest price.

### *5. Report on Orders Above the Average Order Amount Using a Subquery*

This query identifies orders whose total amounts are greater than the average amount of all orders.

```
SELECT
    Order_ID,
    Customer_ID,
    Total_Amount,
    Order_Status
FROM ORDERS
WHERE Total_Amount > (
    SELECT AVG(Total_Amount)
    FROM ORDERS
)
ORDER BY Total_Amount DESC;
```

| Order_ID | Customer_ID | Total_Amount | Order_Status |
|---|---|---|---|
| 404 | 4 | 280.00 | Preparing |
| 401 | 1 | 260.00 | Delivered |
| NULL | NULL | NULL | NULL |

The subquery calculates the average total order amount, which is ₹194. The outer query returns only orders above this average. This report helps identify higher-value orders.

### *6. Report on the Highest-Rated Restaurant Using a Subquery*

This query identifies the restaurant or restaurants with the highest rating in the database.

```
SELECT Restaurant_ID, Restaurant_Name, Rating
FROM RESTAURANT
WHERE Rating = (
   SELECT MAX(Rating)
   FROM RESTAURANT
);
```

| | Restaurant_ID | Restaurant_Name | Rating |
|---|---|---|---|
| ▶ | 104 | Urban Tandoor | 4.7 |
| * | NULL | NULL | NULL |

The inner query finds the maximum restaurant rating. The outer query displays the restaurant whose rating matches that value. This report helps identify the highest-rated restaurant in the system.

## Conclusion

In this assignment, advanced SQL queries were implemented using INNER JOIN, LEFT JOIN, and subqueries. The reports combined information from related tables and provided useful details about customer orders, ordered food items, restaurant menus, order amounts, and restaurant ratings. These queries demonstrate how joins and subqueries can retrieve related records and generate meaningful reports for the Food Delivery Management System.

| | | | | | |
|---|---|---|---|---|---|
| ✓ | 62 | 12:02:08 | SELECT O.Order_ID, C.Customer_Name, R.Restaurant_Name, O.Order_... | 5 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ | 63 | 12:04:58 | SELECT O.Order_ID, C.Customer_Name, M.Item_Name, OI.Quantity, ... | 6 row(s) returned | 0.016 sec / 0.000 sec |
| ✓ | 64 | 12:06:20 | SELECT R.Restaurant_ID, R.Restaurant_Name, COUNT(M.Item_ID) AS T... | 5 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ | 65 | 12:07:24 | SELECT Item_ID, Item_Name, Price FROM MENU_ITEM WHERE Price > ( SEL... | 4 row(s) returned | 0.015 sec / 0.000 sec |
| ✓ | 66 | 12:08:13 | SELECT Order_ID, Customer_ID, Total_Amount, Order_Status FROM O... | 2 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ | 67 | 12:09:03 | SELECT Restaurant_ID, Restaurant_Name, Rating FROM RESTAURANT WHER... | 1 row(s) returned | 0.000 sec / 0.000 sec |